QueryAPIGate BLOG · BENCHMARK

The 30% that was in the build

QueryAPIGate's Docker image ran Python about 30% slower per CPU than other builds of the same Python. We found why, measured what fixing it is worth at the gateway (less than 30%), found that the obvious fix came with a security cost, and chose the one that didn't.

Published 31 October 2026 · Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?p=the-30-percent-in-the-build

When we shipped the Rust data plane in QueryAPIGate's Docker image, we measured the image against a build from source, expecting to prove that containers cost nothing. They didn't. But the measurement found something else: the Python inside the image ran about 30% slower per CPU than the Python on the same laptop. Same workload, same core, same package versions. Seccomp on or off made no difference.

Every call Python handles in the image pays that: Python serving alone, and everything the data plane hands back to it to record. So we spent a day finding out where the 30% went, what getting it back is worth, and what it would cost. This post is that day: what we measured, where the obvious answer was wrong, and why we chose what we chose.

Finding the cause

First, the candidates. One workload parses batches of reported runs, as QueryAPIGate's recording does; the other is a plain Python loop with no library at all. Each was pinned to one core and run in five rotated rounds. Below 1 is faster than the image's interpreter:

INTERPRETER	PYTHON	RECORDING WORKLOAD	PLAIN LOOP
The image's base, the official `python:3.12-slim`	3.12.15	1.00	1.00
Official `python:3.13-slim`	3.13.18	1.01	1.09
Official `python:3.14-slim`	3.14.10	1.12	1.17
Debian trixie's own `python3`	3.13.5	0.73	0.70
python-build-standalone	3.13.18	0.75	0.68
Ubuntu 24.04's `python3.12`	3.12.3	0.79	0.78

Every official image was the slow one, and a newer Python didn't help: 3.14-slim was slower again. Every statically linked interpreter, from three different builders, was 21-36% faster.

So the interpreter wasn't slow; the build was. To prove it, we rebuilt CPython 3.12.15 from the official image's own recipe, read out of its `docker history`. The rebuild measured the same as the image (1.01). Then we took one setting out at a time:

- a statically linked interpreter, instead of one loading `libpython3.12.so`, gained 8–16%;
- leaving out the frame pointers gained 2–4%;
- leaving out the Debian build flags the recipe appends to CPython's own gained another 7–10%.

There's one thing we can't explain. Debian builds its own fast Python with the same `-O2` and the same hardening flags, so no single flag is the villain. We stopped taking it apart there. Knowing the gap is in the build, and that other builds don't have it, was enough to decide on.

What it's worth at the gateway: less than 30%

An interpreter that's 27% faster doesn't make an API gateway 27% faster. Much of a call's CPU isn't the interpreter's. The PostgreSQL driver, the C extensions and the kernel's networking are the same code in every image. So we measured the whole thing. We built both images from the same application (the diff was empty) and ran them as the Helm chart does, interleaved on the same eight cores, against PostgreSQL with 10 million rows.

With Python serving calls itself, one worker as the image runs by default, the new image served **6–9% more calls a second per core**, with p99 latency 7% lower. That held in every round and every scenario: plain lookups, masked and table-restricted lookups, and 100-row pages.

With the data plane in front, the Rust process serves the calls and Python records each run behind it. The calls per second didn't change; that's the data plane's work. But recording is pure Python, the kind of loop the interpreter speeds up most. Past its ceiling, at 28,000 calls a second, one Python worker recorded **67% of the runs instead of 42%**, with 5% less CPU. Where every run was recorded, it needed **20% less CPU** to do it.

So here is the honest summary. For a gateway that runs Python alone, it's a modest gain that every call gets. For one behind the data plane, it's headroom where we needed it most: the audit trail.

The obvious fix, and its cost

The obvious fix was Debian's own Python. It was as fast as anything we measured. Every library under it is a Debian package, which Debian patches and every container scanner understands. Dependabot already updates the base image. We wrote the design decision around it, built it, and the whole test suite passed inside it.

Then we scanned it. Trivy found **273 issues in the image's Debian packages, 70 of them HIGH**. Today's image has 166, with 44 HIGH. The difference was Debian's Python itself. Debian trixie ships Python 3.13.5 for the life of the release, with the fixes Debian chooses to backport. On the day we checked, 19 CVEs were open against it, 5 of them HIGH, and upstream's later 3.13 releases fix them. One of them, `SSLContext.wrap_bio()` skipping hostname verification when no hostname is given, we checked by behaviour rather than trusting a database:

PYTHON	`WRAP_BIO()` WITH NO HOSTNAME
Debian's 3.13.5	accepted, silently
Today's official 3.12.15	accepted, with a deprecation warning
Upstream 3.13.18	refused

This last check also taught us something about today's image. Its Python is compiled into `/usr/local`, where scanners don't look, so its clean-looking score wasn't a clean bill of health. Debian's Python looked worse partly because, for once, the scanner could see it.

We weren't willing to ship 26 new HIGH findings to the people who scan our image before running it, even with a good explanation for each one. So we measured the alternative.

What we chose: upstream's Python, built by python-build-standalone

python-build-standalone publishes upstream CPython releases as statically linked builds, optimized with PGO, LTO and BOLT. It's what `uv` installs when you ask it for a Python. We measured it the same way, image against image, against Debian's:

- **speed:** within 1-2% of Debian's, in every scenario;
- **tests:** the whole suite passes inside the image, on SQLite and on the PostgreSQL store, with Redis;
- **scanner:** 164 findings in Debian's packages, 43 HIGH, one fewer than today's image;
- **Python:** upstream's latest 3.13 release, now 3.13.18, with upstream's security fixes.

The supply chain matters as much as the speed, so here's how it's pinned:

- **`uv` is a pinned build stage**, `FROM ghcr.io/astral-sh/uv:<version>@sha256:...`, like every base image we use. Dependabot proposes its updates; a test checks that nothing is ever copied from an image that isn't a stage, where Dependabot couldn't see it.
- **The `uv` pin is the Python pin.** Each `uv` release carries the list of Python releases it knows, with their hashes, and resolves "3.13" from that list without asking the network: `uv` 0.13.6 gives 3.13.18, 0.13.0 gave 3.13.16. A Dependabot bump of `uv` brings Python's next patch release, and `uv` checks the download's hash.
- **Only what runs is shipped.** Tk, IDLE, the install's own `pip` and the unused shared `libpython` stay behind in the build stage. The image is 465 MB, 21.5 MB more than today's.
- **The base stays Debian trixie,** pinned by digest, the release the Rust data plane is compiled against.

What a scanner sees

If you scan our image before you run it, and you should, here's what changes:

- **Debian packages:** the same as before, 164 findings against 166.

- **Python itself:** still not a Debian package, so still not something a scanner reports by version, as before. The difference is that it's now upstream's current release, not one frozen at a distribution's.
- **Python's `ssl`:** it now uses the OpenSSL compiled into the Python build (3.5.9, currently newer than Debian's 3.5.7). The image already worked this way for its most-used crypto: the PostgreSQL driver loads its own `libssl`, and `cryptography` carries its own OpenSSL.
- **The virtualenv's `pip`** reports five HIGH findings, for the old copies of urllib3, msgpack and setuptools it vendors. QueryAPIGate never imports them. They're there because `pip` is, so an image built `FROM` ours can add a driver.

Everything else about a release is unchanged. It's signed keyless by digest, and it carries SLSA provenance, an SBOM and a GitHub attestation, all verified in CI before the release job ends.

What we didn't do

- **We didn't measure arm64.** We don't have arm64 hardware. The release workflow builds it and now checks, under emulation, that the arm64 image's Python starts and imports the application. The speed-up's main ingredients (static linking, PGO, LTO) are the same there; BOLT is x86-64 only.
- **We didn't move to Python 3.14.** Its standalone build was the fastest in the plain loop. That's a change of its own, measured on its own.
- **We didn't touch the PyPI package.** `pip install queryapigate` runs on whatever Python you bring, 3.11 to 3.14, as before. If your organization requires a distribution-supplied interpreter, that path is unchanged: install the package on your own base image.

The numbers

The decision is ADR 0031. Every number in this post, the raw results, the rebuild recipes, the scans and the scripts that produced them are in benchmarks/results/python-image-2026-10-31. If you get different numbers on your hardware, we'd like to see them.

READ NEXT
Every run, recorded

QUESTIONS OR PROBLEMS
Discord · GitHub issues

LATEST VERSION OF THIS PAGE
anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?p=the-30-percent-in-the-build

FOLLOW QUERYAPIGATE GitHub X Discord Bluesky Mastodon YouTube Reddit

@queryapigate on X, Bluesky, Mastodon and YouTube · r/queryapigate on Reddit · github.com/AnanthaRajuC/QueryAPIGate

QueryAPIGate · one governed gate between your databases and everything that asks them for data · self-hosted, Apache-2.0 · © 2026 QueryAPIGate